CS52 · AI & ML · CIE-1

Predicted Questions & Model Answers

20 high-probability 5-mark questions across Units I–III, with full answers written the way examiners want, plus memory tricks for each one.

Unit I — 7 Qs

PEAS · A* on 8-puzzle · Propositional-logic argument · Agent types · CNF conversion · Environment types · Choosing a search strategy

Unit II — 7 Qs

"Curiosity killed the cat" · ML case-based (which learning?) · Bayes reasoning · John is happy · John likes peanuts · Well-posed problems · Find-S

Unit III — 6 Qs

CR's 3 questions first: classification vs regression · Naive Bayes · Linear regression with example — then 3 application problems

Built from: last year's CIE-1 paper (27/11/2024), the insider topic list, your sir's hint (scenario / problem-solving, no pseudocode), and your CR's Unit III list.

How this paper will look — and how to read this PDF

Last year's CIE-1 pattern (30 marks, 1 hour)

Q	Last year	Marks	Unit
1a	Distinguish learning agent vs goal-based agent	5	I
1b	Check validity of an argument using propositional logic	5	I/II
1c	5 scenarios → supervised / unsupervised / reinforcement?	5	II
2a	A* on 8-puzzle, g = depth, h = misplaced tiles	8	I
2b	John likes peanuts: FOL + backward chaining	7	II
3a	Convert two formulas to CNF	7	I/II
3b	"Is John happy?" by resolution refutation	8	II

Q1 is compulsory (3 × 5 marks); then answer **Q2 or Q3** (7 + 8 marks). So you need: all three 5-markers + **one** full choice block.

What your sources point to

Source	Says	Mapped to
Insider list	Machine learning → case-based example	II-Q2 SURE
Insider list	Curiosity killed the cat	II-Q1 SURE
Insider list	A*, propositional logic	I-Q2, I-Q3 SURE
Insider list	Bayesian theorem reasoning	II-Q3 SURE
Insider list	PEAS	I-Q1 SURE
Your sir	Scenario / problem-solving, no algorithms or pseudocode	Every answer below is scenario-first
Your CR	Classification vs regression; Naive Bayes; Linear regression with example	III-Q1–Q3 SURE

PREDICTION FOR THIS YEAR

Q1 (compulsory 5-markers): PEAS scenario · propositional-logic reasoning · ML case-based identification. **Q2/Q3 (problem solving):** A* · Curiosity-killed-the-cat resolution · Bayes reasoning · CNF / chaining. Unit III: the CR's three questions.

Labels used on every question

SURE SHOT = named in the insider list / CR list, or repeated pattern from last year — prepare word-perfect.

VERY LIKELY = came last year or is the natural variant of a sure-shot topic.

IMPORTANT TOPIC = not confirmed, but fits the "scenario" hint — study the idea, not word-for-word.

HOW TO WRITE A 5-MARK ANSWER

1. **1 mark** — definition / setup (symbols, givens).
2. **2 marks** — the working: table, steps or diagram.
3. **1 mark** — the example applied to the scenario.
4. **1 mark** — conclusion in a box + one line "why".

Aim for ~1 page. Always draw something (table, tree, diagram).

HOW TO REMEMBER THINGS (WORKS FOR THIS SUBJECT)

- **Draw, don't read:** redraw each diagram once from memory — drawing sticks 3× better than rereading.
- **Mnemonics** for lists (each question has one).
- **Story method** for logic problems: picture the story (Jack, his dog, Tuna the cat) — the clauses follow the story order.
- **Active recall:** cover the answer, write it, then check. Do this for every SURE SHOT.
- **Spaced repetition:** revise SURE SHOTs tonight, tomorrow morning, and 1 hour before the exam.
- **Teach it:** explain A* or Bayes aloud to a friend in 2 minutes — gaps show up instantly.

I Unit I — Agents, Search & Logic (theory + one big problem)

Q1. Give the PEAS description for a self-driving car (or any given agent) and classify its environment. SURE SHOT

Why: "PEAS" is on the insider list; last year's Q1 had an agent question. Expect a fresh scenario: self-driving car, food-delivery drone, medical-diagnosis system, robot vacuum, chess bot, smart home assistant.

DEFINITION (1 MARK)

PEAS = **P**erformance measure, **E**nvironment, **A**ctuators, **S**ensors. It specifies the task environment of an agent — the first step in designing any agent. Performance = how success is judged; Environment = where it works; Actuators = how it acts; Sensors = how it perceives.

PEAS	Self-driving car (automated taxi)
Performance	Safe (no accidents), fast / shortest time, legal (obeys traffic rules), comfortable trip, fuel-efficient, maximise profit / passenger satisfaction
Environment	Roads, other vehicles, pedestrians, cyclists, traffic signals, weather, potholes, passengers
Actuators	Steering, accelerator, brake, gear, horn, indicators, headlights, display / speaker to passenger
Sensors	Cameras, LIDAR, RADAR, GPS, speedometer, odometer, accelerometer, engine sensors, microphone / touchscreen

Environment type (1 mark): partially observable (can't see around corners), multi-agent (other drivers), stochastic (unpredictable traffic), **sequential** (each action affects the future), **dynamic** (world changes while deciding), **continuous** (speed, position), known. → the hardest kind of environment.

Agent	Performance	Environment	Actuators	Sensors
Medical diagnosis	Healthy patient, low cost, fewer lawsuits	Patient, hospital, staff	Display questions, tests, diagnoses, treatments	Keyboard entry of symptoms, test results, patient answers
Robot vacuum	Cleanliness, battery use, time, no damage	Rooms, floor, furniture, dirt, people, pets	Wheels, brushes, suction, bumper	Dirt sensor, bump sensor, cliff sensor, camera
Food-delivery drone	On-time delivery, safety, battery, intact food	Sky, buildings, weather, birds, customers	Propellers, gripper / drop mechanism	GPS, camera, altimeter, wind sensor
Chess bot (with clock)	Win / draw / loss, time used	Board, opponent, clock	Move pieces (display move)	Board state, clock
Wumpus-world agent	+1000 gold out, −1000 death, −1/step, −10 arrow	4×4 cave, wumpus, pits, gold	Forward, TurnLeft/Right, Grab, Shoot, Climb	Stench, Breeze, Glitter, Bump, Scream
English tutor	Students' test scores	Students, testing agency	Exercises, suggestions, corrections	Typed words / speech

REMEMBER IT

"P-E-A-S = Purpose, Every-thing around, Arms, Senses." Actuators = what it **does** (verbs: steer, brake). Sensors = what it **feels** (nouns: camera, GPS). Never put "camera" under actuators — that's the most common mistake.

Q2. Apply A* to solve the 8-puzzle (g = depth, h = number of misplaced tiles).

SURE SHOT

Why: last year's Q2a (8 marks) + "A*" on the insider list. Same method works for any start/goal they give.

Start (last year's paper)

2	8	3
1	6	4
7		5

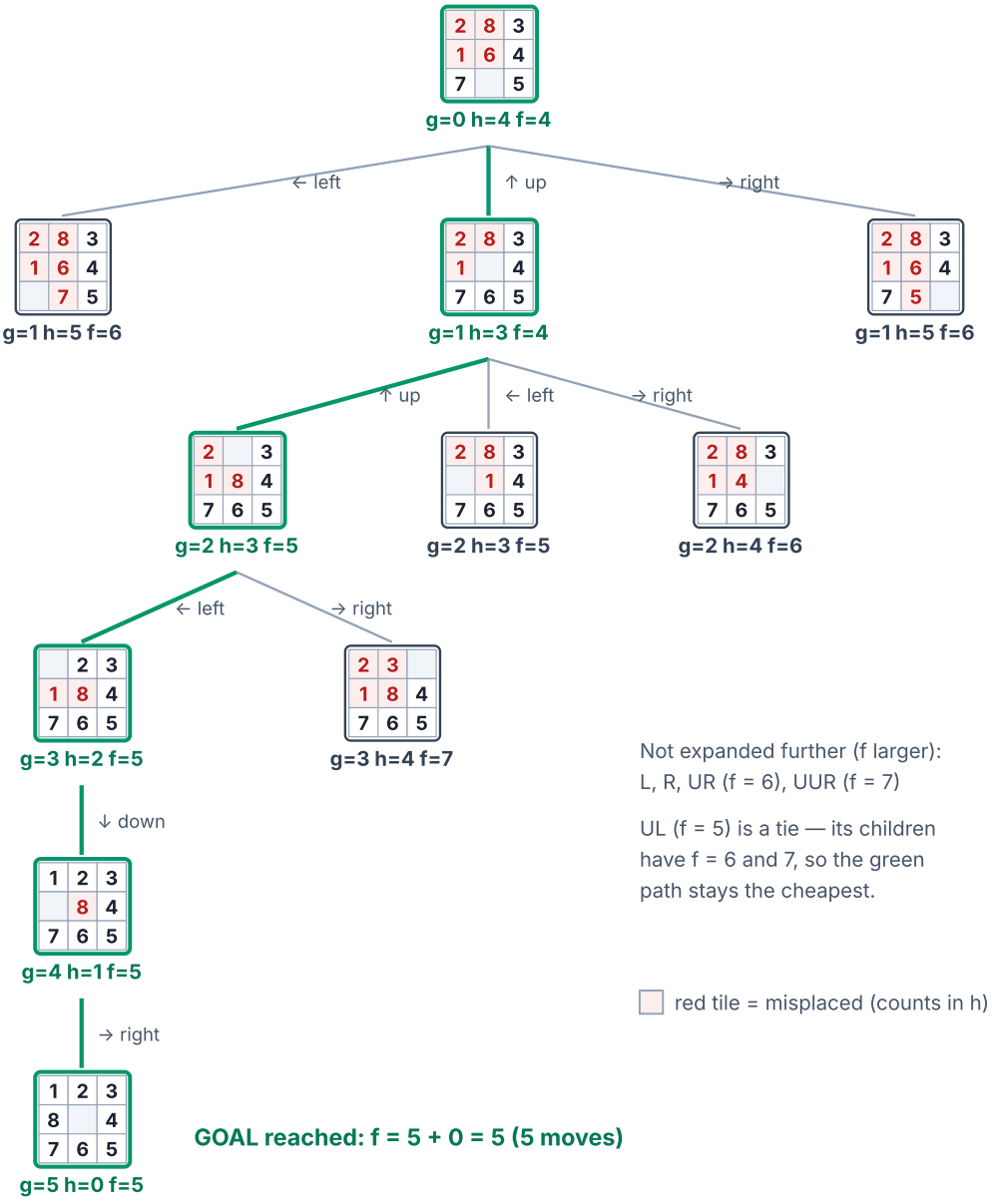
Goal

1	2	3
8		4
7	6	5

2 8 3 / 1 6 4 / 7 _ 5 → 1 2 3 / 8 _ 4 / 7 6 5

A* IN ONE LINE

Expand the node with the lowest f(n) = g(n) + h(n). g = moves made so far (depth); h = misplaced tiles (don't count the blank). Admissible h ⇒ A* finds the optimal (cheapest) path.



Search tree. Moves are named by where the blank goes. Green = chosen path; every board shows g, h, f.

Step	Expand (lowest f)	Children generated (f = g + h)	Open list after
0	Start: h = 4 (tiles 2, 8, 1, 6 misplaced), f = 0 + 4 = 4	L: 1+5 = 6, U: 1+3 = 4, R: 1+5 = 6	U(4), L(6), R(6)
1	U (f = 4)	UU: 2+3 = 5, UL: 2+3 = 5, UR: 2+4 = 6	UU(5), UL(5), L, R, UR(6)
2	UU (f = 5, tie → pick first)	UUL: 3+2 = 5, UUR: 3+4 = 7	UUL(5), UL(5), ...
3	UUL (f = 5)	UULD: 4+1 = 5	UULD(5), UL(5), ...
4	UULD (f = 5)	GOAL: 5+0 = 5	GOAL(5)
5	**Goal tested → stop.** Path: blank Up → Up → Left → Down → Right. **Cost = 5 moves** (optimal).		

EXAM TIPS FOR A*

- Write **f = g + h** and state h = misplaced tiles **excluding the blank** at the top.
- Under every board write g, h, f. Circle the node you expand next — that shows "lowest f first".
- Don't expand back to the parent state (repeated state).
- End with: path, number of moves, "optimal because h is admissible (never overestimates)".
- If they say h = Manhattan distance instead, only the h numbers change — method is identical.

REMEMBER IT

"Go = Gone + Guess." f (total) = g (steps **gone**) + h (**guess** to goal). Pick the smallest total every time — like choosing the auto with the lowest meter + remaining-distance estimate.

Q3. "If the team works overtime, they will finish on time. The team worked overtime. Therefore they finished on time." Check validity using propositional logic. Is overtime necessarily sufficient? SURE SHOT

Why: last year's Q1b + "propositional logic" on the insider list. Expect the same type with a new story (rain/match, study/pass, alarm/burglar).

Symbols: O = "the team works overtime", F = "the team finishes on time". **Premises:** P1: O ⇒ F, P2: O. **Conclusion:** F.

O	F	O ⇒ F (P1)	O (P2)	F (concl.)
T	T	T	T	**T ← only row with both premises true**
T	F	F	T	F
F	T	T	F	T
F	F	T	F	F

Valid — in every row where all premises are true, the conclusion is true. This is the inference rule **Modus Ponens**: $O \Rightarrow F,\ O \vdash F$. Equivalently $(O \Rightarrow F) \wedge O \Rightarrow F$ is a **tautology**.

Is overtime necessarily sufficient? No — validity is about form, not truth. The argument only shows: *if* premise 1 is true, the conclusion must follow. Whether overtime really guarantees completion is a real-world claim (scope creep, bugs, sick staff can still delay it). Valid + true premises = **sound**; we can't prove premise 1 by logic alone.

THE TRAP VARIANTS (LIKELY TWIST THIS YEAR)

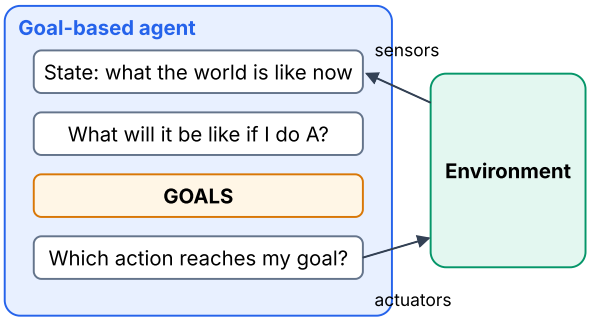
Form	Example	Valid?
Modus Ponens: P⇒Q, P ⊢ Q	Overtime ⇒ on time; overtime ⊢ on time	**Valid**
Modus Tollens: P⇒Q, ¬Q ⊢ ¬P	...; not on time ⊢ no overtime	**Valid**
Affirming the consequent: P⇒Q, Q ⊢ P	...; finished on time ⊢ they worked overtime	**Invalid (row O=F, F=T)**
Denying the antecedent: P⇒Q, ¬P ⊢ ¬Q	...; no overtime ⊢ not on time	**Invalid (row O=F, F=T)**

REMEMBER IT

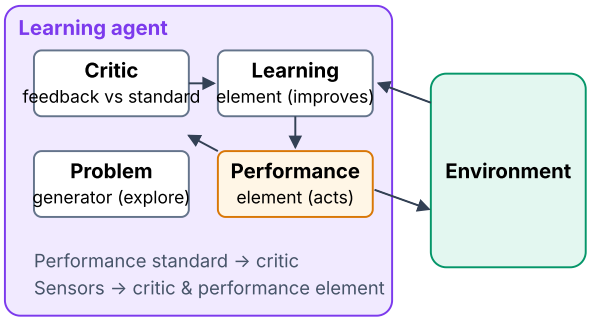
"**P ⇒ Q is false only when Promise is broken**": P true, Q false. For validity: **find a row where premises are all T but conclusion is F**. Found one → invalid. None → valid. Mnemonic for valid forms: "**Ponens Pushes forward, Tollens Turns back**".

Q4. Distinguish between a learning agent and a goal-based agent (with diagrams and an example). VERY LIKELY

Why: exact Q1a last year. This year they may swap the pair (simple reflex vs model-based, goal-based vs utility-based) — the table at the end covers every pair.



Goal-based: plans/searches for actions that reach a goal.



Learning: improves its own behaviour from feedback.

Basis	Goal-based agent	Learning agent
Idea	Chooses actions that achieve a desired **goal**, using search and planning	Improves its performance over time from **experience / feedback**
Key parts	State + model of the world + goal information	Performance element, critic, learning element, problem generator
Knowledge	Fixed — given by the designer	Grows — modifies itself; can start with little knowledge
Handles new situations	Only if the model covers them	Adapts to unknown environments
Question it asks	"Which action leads to my goal?"	"How can I do better next time?"
Example	Google Maps routing to a destination; taxi deciding a route to the airport	Spam filter improving from user "report spam"; taxi learning that braking hard on wet roads is bad

Note: any agent type (including goal-based) can be turned into a learning agent — the learning element improves its performance element.

Other pairs they may ask (one-line differences)	
Simple reflex vs model-based	Reflex acts only on the **current percept** with if-then rules (fails in partially observable worlds); model-based keeps an **internal state** using a model of how the world changes.
Goal-based vs utility-based	Goal: success is binary (reached / not); utility: a **utility function** scores how good each state is (fastest, safest, cheapest route) — chooses the best among many ways to reach the goal.

REMEMBER THE 5 AGENT TYPES IN ORDER (EACH ADDS ONE THING)

"Silly Monkeys Grab Useful Lemons" — Simple reflex (rules) → Model-based (+ memory/state) → Goal-based (+ goal) → Utility-based (+ happiness score) → Learning (+ improves itself). Learning parts: "**C-L-P-P**" = Critic, Learning element, Performance element, Problem generator.

Q5. Convert to CNF: (a) (A ∨ (B ∧ C)) ⇒ (¬D ∨ E) (b) (A ∨ (B ∧ (C ∨ ¬D))) ∧ (¬E ⇒ F) VERY LIKELY

Why: last year's Q3a (7 marks) and part of every resolution question. Practise until it's mechanical.

4 STEPS (ALWAYS IN THIS ORDER)

① Remove ⇔ : P⇔Q → (P⇒Q) ∧ (Q⇒P) ② Remove ⇒ : P⇒Q → ¬P ∨ Q ③ Push ¬ inside (De Morgan, ¬¬P = P) ④ Distribute ∨ over ∧ : P ∨ (Q ∧ R) → (P ∨ Q) ∧ (P ∨ R)

(a)

(A ∨ (B ∧ C)) ⇒ (¬D ∨ E)

② ¬(A ∨ (B ∧ C)) ∨ (¬D ∨ E)

③ (¬A ∧ ¬(B ∧ C)) ∨ ¬D ∨ E

③ (¬A ∧ (¬B ∨ ¬C)) ∨ ¬D ∨ E

④ distribute (¬D ∨ E) over ∧:

(¬A ∨ ¬D ∨ E) ∧ (¬B ∨ ¬C ∨ ¬D ∨ E)

(b)

(A ∨ (B ∧ (C ∨ ¬D))) ∧ (¬E ⇒ F)

② ¬E ⇒ F → ¬¬E ∨ F → E ∨ F

④ A ∨ (B ∧ (C ∨ ¬D)) → (A ∨ B) ∧ (A ∨ C ∨ ¬D)

Combine:

(A ∨ B) ∧ (A ∨ C ∨ ¬D) ∧ (E ∨ F)

3 clauses: {A, B}, {A, C, ¬D}, {E, F}

PRACTICE VARIANT (BICONDITIONAL) — TRY IT, ANSWER BELOW

B ⇔ (P ∨ Q) → (¬B ∨ P ∨ Q) ∧ (¬P ∨ B) ∧ (¬Q ∨ B).

REMEMBER IT

"I-I-N-D" = "I Intend No Dumb mistakes": **I**ff out → **I**mplies out → **N**egation in → **D**istribute. De Morgan: "**break the line, change the sign**" (¬ over ∧ becomes ∨ and vice versa).

Q6. Classify the task environment of the given agents (fully/partially observable, deterministic/stochastic, etc.) IMPORTANT TOPIC

Why: usually attached to PEAS (sir likes scenarios). 5 marks = define dimensions + fill a table.

Dimension	Test question to ask
Fully vs partially observable	Can the sensors see the complete relevant state?
Single vs multi-agent	Are other agents affecting my performance? (competitive / cooperative)
Deterministic vs stochastic	Is the next state fully decided by the current state + my action?
Episodic vs sequential	Does the current decision affect future decisions?
Static vs dynamic	Can the world change while I'm thinking? (semi-dynamic: only the score/clock changes)
Discrete vs continuous	Finite distinct states/actions, or smooth values?
Known vs unknown	Does the agent know the rules/outcomes of its actions?

Task	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess with a clock	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Taxi / self-driving car	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Image analysis	Fully	Single	Deterministic	Episodic	Semi	Continuous
Part-picking robot	Partially	Single	Stochastic	Episodic	Dynamic	Continuous
Wumpus world	Partially	Single	Deterministic	Sequential	Static	Discrete
Vacuum cleaner (2 rooms)	Partially (local dirt sensor)	Single	Deterministic	Episodic	Static	Discrete

REMEMBER IT

"**O-A-D-E-S-D-K**" → "**O**ld **A**gents **D**on't **E**at **S**picy **D**osa, **K**id" = Observable, Agents, Deterministic, Episodic, Static, Discrete, Known. Real world = the "hard" side of every pair.

Q7. Scenario: which search strategy would you use and why? (BFS / DFS / UCS / A*) IMPORTANT TOPIC

Why: sir said "problem solving, no pseudocode" — so search comes as "choose and justify", not "write the algorithm".

Scenario	Best choice	Justification
Find the route with the fewest metro stops	**BFS**	All steps cost the same (1 stop) → BFS gives the shallowest = optimal path; complete.
Delivery robot: roads have different lengths; need cheapest route; no distance estimate	**Uniform-cost search**	Expands lowest path cost g(n) first → optimal for unequal costs.
Google-Maps-like routing where straight-line distance to destination is known	**A***	f = g + h uses the admissible straight-line heuristic → optimal and far fewer nodes than UCS.
Solving a maze on a phone with very little memory; depth limited	**DFS / Iterative deepening**	DFS memory O(bm); IDS adds completeness and optimality (equal costs) with O(bd) memory.
Need a quick, "good enough" answer, optimality not required	**Greedy best-first**	f = h only → fast, but not optimal (Arad→Bucharest gives 450 km instead of 418).
Start and goal both known, uniform costs (e.g., friend-of-friend link)	**Bidirectional**	Searches from both ends: $O(b^{d/2})$ instead of $O(b^d)$.

Property	BFS	UCS	DFS	IDS	Greedy	A*
Complete?	Yes	Yes	No	Yes	No	Yes
Optimal?	Equal costs	Yes	No	Equal costs	No	Yes (admissible h)
Uses	queue	g(n)	stack	depth limits	h(n)	g(n)+h(n)

REMEMBER IT

BFS = **B**readth = layer by layer ("ripples in a pond"). **D**FS = **D**ive deep ("go down one corridor first"). UCS = "**U**ber **C**heapest **S**o far". Greedy = "looks only ahead". A* = "looks back (g) **and** ahead (h)".

UNIT I — ALSO GLANCE AT (1-LINE ANSWERS)

Rational agent = does the action that maximises expected performance given its percept sequence and knowledge (rational ≠ omniscient). Turing test = interrogator can't tell machine from human (needs NLP, knowledge representation, reasoning, learning). Admissible h = never overestimates; consistent $h(n) \le c(n, n') + h(n')$.

II Unit II — Logic Solving, Bayes & Intro to ML

Q1. "Curiosity killed the cat" — prove using resolution. SURE SHOT

Why: named directly on the insider list. It's the classic textbook resolution problem — learn it fully (it can carry 7–8 marks).

> ***QUESTION***
>
> *Jack owns a dog. Every dog owner is an animal lover. No animal lover kills an animal. Either Jack or Curiosity killed the cat, who is named Tuna. Did Curiosity kill the cat?*

Step 1 — English → FOL

A. ∃x Dog(x) ∧ Owns(Jack, x)

B. ∀x (∃y Dog(y) ∧ Owns(x, y)) ⇒ AnimalLover(x)

C. ∀x AnimalLover(x) ⇒ (∀y Animal(y) ⇒ ¬Kills(x, y))

D. Kills(Jack, Tuna) ∨ Kills(Curiosity, Tuna)

E. Cat(Tuna)

F. ∀x Cat(x) ⇒ Animal(x) (background: cats are animals)

Goal: Kills(Curiosity, Tuna)

Step 2 — CNF (clause form)

A1. Dog(D) A2. Owns(Jack, D) (EI: Skolem constant D)

B. ¬Dog(y) ∨ ¬Owns(x, y) ∨ AnimalLover(x)

C. ¬AnimalLover(a) ∨ ¬Animal(b) ∨ ¬Kills(a, b)

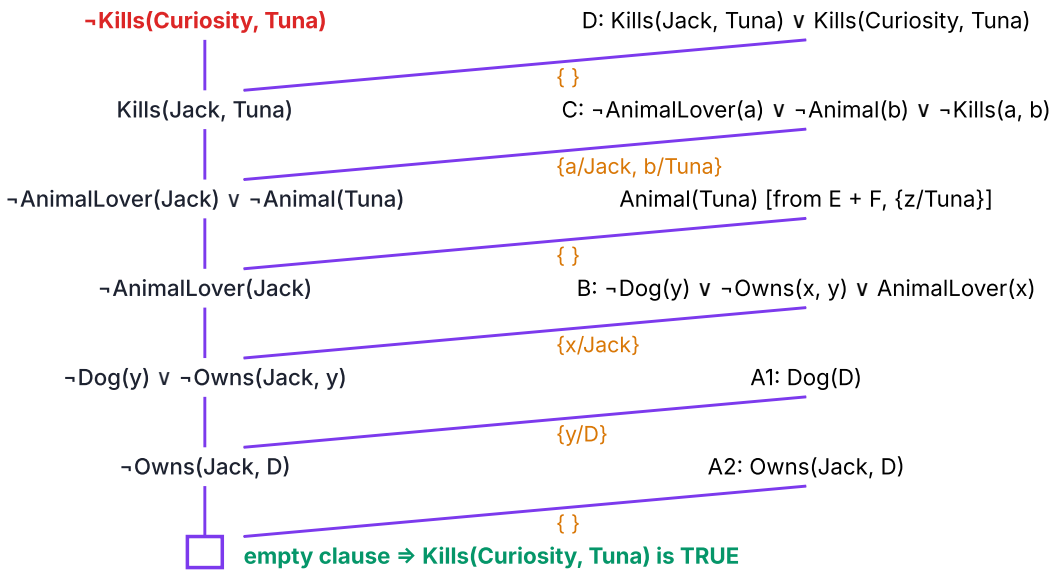
D. Kills(Jack, Tuna) ∨ Kills(Curiosity, Tuna)

E. Cat(Tuna)

F. ¬Cat(z) ∨ Animal(z)

Step 3 — negate goal: ¬Kills(Curiosity, Tuna)

How B became a clause: (∃y Dog(y) ∧ Owns(x, y)) ⇒ AnimalLover(x) → ¬∃y(...) ∨ ... → ∀y (¬Dog(y) ∨ ¬Owns(x, y)) ∨ AnimalLover(x). **C:** ¬AnimalLover(x) ∨ ∀y(¬Animal(y) ∨ ¬Kills(x, y)), variables renamed a, b (standardize apart).



Step 4 — resolution graph (left: resolvents; right: clause used; orange: substitution). "Animal(Tuna)" first comes from resolving E with F using {z/Tuna}.

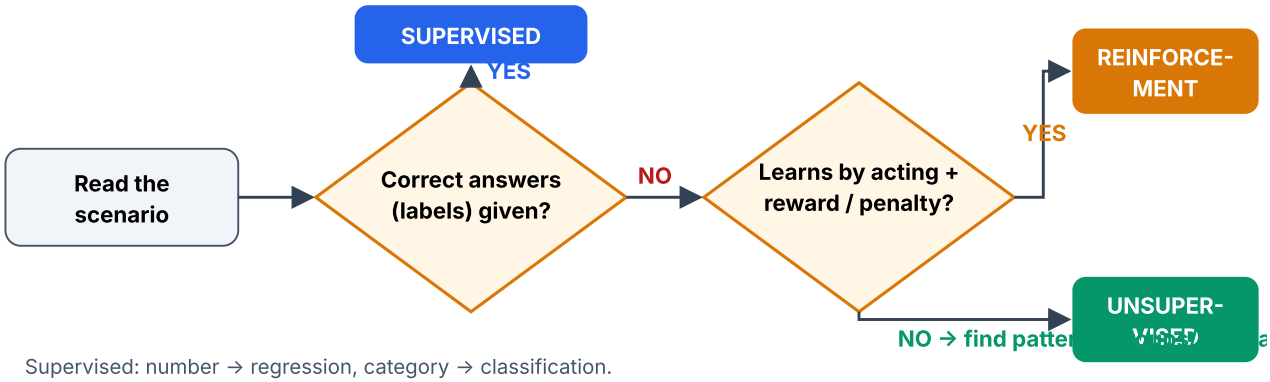
Conclusion: ¬Kills(Curiosity, Tuna) leads to a contradiction, so **Curiosity killed the cat**. (Reason in English: Jack owns a dog → Jack is an animal lover → he can't kill an animal → Tuna is a cat, so an animal → Jack didn't kill Tuna → by D, Curiosity did.)

REMEMBER IT — THE STORY CHAIN

Say it as one breath: "**Dog → Lover → No-kill → Cat is Animal → Not Jack → Curiosity**." Resolution just walks this chain backwards from ¬Kills(Curiosity, Tuna). Six resolve steps; the two Skolem facts Dog(D), Owns(Jack, D) are used last.

Q2. Identify the type of learning (supervised / unsupervised / reinforcement) in each scenario and justify. SURE SHOT

Why: last year's Q1c (5 marks) + "Machine Learning → case based example" on the insider list. Expect 5 new scenarios — use the flowchart.



Supervised: number → regression, category → classification.

Decision flowchart — ask two questions and you're done.

Scenario (last year's paper)	Type	One-line justification
A. Hospital uses patient records with labelled diagnoses to predict diseases in new patients	**Supervised (classification)**	Labelled examples (record → diagnosis) are given
B. Game AI maximises chess score by trying strategies, learning from wins/losses/draws	**Reinforcement**	Learns by trial and error from reward (win) / penalty (loss)
C. Company analyses sales data to detect anomalies (sudden drops/spikes) without knowing causes	**Unsupervised** (anomaly detection)	No labels; finds unusual patterns
D. Self-driving car learns decisions by interacting with a simulator and receiving rewards/penalties	**Reinforcement**	Agent–environment interaction + rewards
E. Customer reviews fed to an algorithm that finds distinct topics; no labels	**Unsupervised** (clustering / topic modelling)	Groups similar reviews without labels

Practice set — likely new scenarios this year		
Spam filter trained on emails marked spam / not spam	Supervised – classification	labels given
Predict house prices from past sales	Supervised – regression	numeric label
Group customers into segments for marketing	Unsupervised – clustering	no labels
"Customers who bought bread also bought butter"	Unsupervised – association	finds co-occurrence
Robot learns to walk by falling and getting penalties	Reinforcement	reward/penalty
Bank flags fraud using past transactions labelled fraud/genuine	Supervised – classification	labels given
Detect unusual network traffic with no attack labels	Unsupervised – anomaly detection	no labels
Thermostat learns to save energy from comfort feedback	Reinforcement	feedback signal, not correct answers
Face unlock trained on labelled photos of the owner	Supervised	labelled images
Compress 100 features into 3 for visualisation	Unsupervised – dimensionality reduction (PCA)	no labels

ANSWER FORMAT (1 MARK EACH)

Write "**A → Supervised.** Because labelled data (diagnosis) is available; the model learns input → output mapping (classification)." Always include the **keyword from the scenario** that decided it ("labelled", "reward", "no prior labels").

REMEMBER IT

Teacher, Explorer, Gamer. Supervised = learns from a **teacher** who gives answers. Unsupervised = an **explorer** who finds groups alone. Reinforcement = a **gamer** who learns from score / reward. Keywords: "labelled" → S; "no labels / patterns / groups / anomalies" → U; "reward / penalty / trial-and-error / interacts" → R.

Q3. Bayes' theorem reasoning: a test is positive — what's the probability the person really has the disease? SURE SHOT

Why: "Bayesian theorem reasoning" on the insider list. Expect a medical / spam / fraud scenario with numbers. Same 4-line method every time.

TYPICAL QUESTION

A disease affects 1% of people. A test detects it 95% of the time when present (sensitivity), but also gives a false positive for 10% of healthy people. A patient tests positive. Find P(Disease | Positive). Should the doctor conclude the patient has the disease?

Given: $P(D) = 0.01$, $P(\neg D) = 0.99$
$P(+|D) = 0.95$, $P(+|\neg D) = 0.10$
Bayes: $P(D|+) = P(+|D)\,P(D) / P(+)$
Total probability: $P(+) = P(+|D)P(D) + P(+|\neg D)P(\neg D)$
$= 0.95 \times 0.01 + 0.10 \times 0.99 = 0.0095 + 0.099 = 0.1085$
$P(D|+) = 0.0095 / 0.1085 =$ **0.0876 ≈ 8.8%**

Reasoning (the "why" mark): even with a positive test there's only ~9% chance of disease, because the disease is **rare** (prior 1%) and false positives from the 99% healthy people outnumber true positives (0.099 vs 0.0095). A strong test doesn't mean a high posterior — the prior matters. The doctor should order a second, confirmatory test.

MAP form (if asked "diagnose"): compare $P(+|D)P(D) = 0.0095$ vs $P(+|\neg D)P(\neg D) = 0.099$ → h_{MAP} = ¬Disease.

Same method, other classic numbers (know the answers)	
Cancer test: P(cancer) = 0.008, P(+|cancer) = 0.98, P(+|¬cancer) = 0.03	0.0078 vs 0.0298 → ¬cancer; P(cancer|+) = 0.21
Meningitis: P(S|M) = 0.7, P(M) = 1/50000, P(S) = 0.01	P(M|S) = 0.0014
Spam: P(S) = 0.2, P(offer|S) = 0.8, P(offer) = 0.4	P(S|offer) = 0.40
Liver disease: P(A) = 0.10, P(B) = 0.05, P(B|A) = 0.07	P(A|B) = 0.14
Face card: P(King | Face)	(1 × 4/52) / (12/52) = 1/3

REMEMBER IT

"Post = Like × Prior ÷ Evidence" ("**P**osting **L**ikes **P**leases **E**veryone"). Steps: **G-B-T-A** — write **G**ivens + complements → **B**ayes formula → **T**otal probability for the denominator → **A**nswer + one-line meaning.

Q4. Using resolution refutation, check whether "John is happy". VERY LIKELY

Why: last year's Q3b (8 marks). If Curiosity comes as the main resolution question, this may come as the alternate choice.

QUESTION

Anyone passing his history exams and winning the lottery is happy. But anyone who studies or is lucky can pass all his exams. John did not study, but John is lucky. Anyone who is lucky wins the lottery.

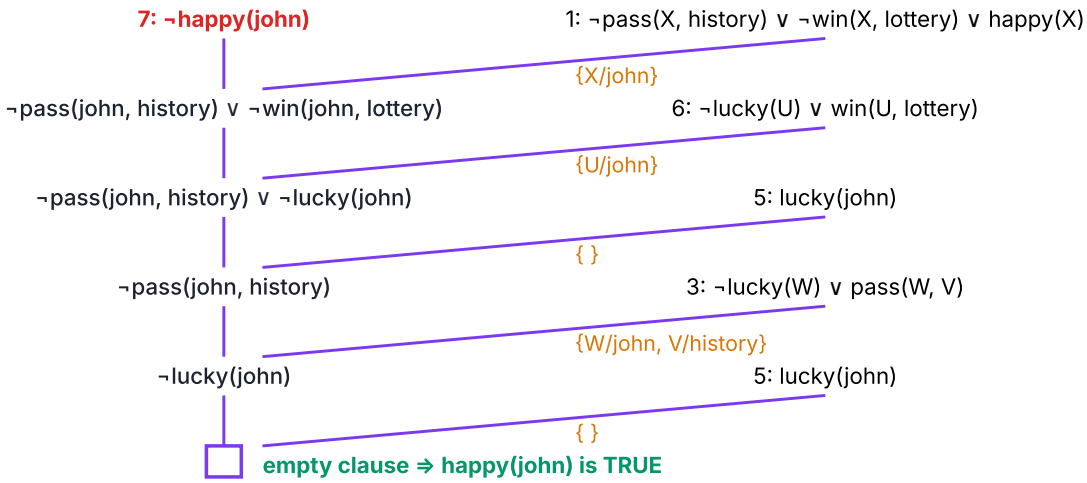
FOL

$\forall X\ pass(X, history) \wedge win(X, lottery) \Rightarrow happy(X)$
$\forall X\ \forall Y\ study(X) \vee lucky(X) \Rightarrow pass(X, Y)$
$\neg study(john) \wedge lucky(john)$
$\forall X\ lucky(X) \Rightarrow win(X, lottery)$

Clauses

1. $\neg pass(X, history) \vee \neg win(X, lottery) \vee happy(X)$
2. $\neg study(Y) \vee pass(Y, Z)$ 3. $\neg lucky(W) \vee pass(W, V)$
4. $\neg study(john)$ 5. $lucky(john)$
6. $\neg lucky(U) \vee win(U, lottery)$
7. **$\neg happy(john)$** (negated goal)

Clauses 2 & 3 come from ¬(study ∨ lucky) ∨ pass → (¬study ∨ pass) ∧ (¬lucky ∨ pass).



REMEMBER IT

"Happy needs Pass + Win; Lucky gives both." So the proof uses lucky(john) **twice** — once to win the lottery, once to pass history. Clause 2 and 4 (study) are a distraction — never used.

Q5. Translate into predicate logic and prove "John likes peanuts" using backward (or forward) chaining. VERY LIKELY

Why: last year's Q2b (7 marks). A repeat or the forward-chaining version is common.

(i) Translation

1. John likes all kinds of food: ∀x Food(x) ⇒ Likes(John, x)
2. Apples are food: Food(Apple)
3. Chicken is food: Food(Chicken)
4. Anything anyone eats and isn't killed by is food: ∀x ∀y Eats(x, y) ∧ ¬Killed(x) ⇒ Food(y)
5. Bill eats peanuts and is still alive: Eats(Bill, Peanut) ∧ ¬Killed(Bill)
6. Sue eats everything Bill eats: ∀x Eats(Bill, x) ⇒ Eats(Sue, x)

```
(ii) Backward chaining — goal first
Goal: Likes(John, Peanut)
 └ rule 1 {x/Peanut} → Food(Peanut)
    └ rule 4 {y/Peanut} → Eats(x, Peanut) ∧
¬Killed(x)
       ├ Eats(x, Peanut) {x/Bill} → fact 5 ✔
       └ ¬Killed(Bill)            → fact 5 ✔
 ⇒ Food(Peanut) ✔ ⇒ Likes(John, Peanut) ✔

Forward chaining (if asked):
 facts → rule 6: Eats(Sue, Peanut)
       → rule 4: Food(Peanut)
       → rule 1: Likes(John, Peanut) ✔
```

EXAM TIPS

Draw the backward-chaining tree **top-down** (goal at top, facts at leaves, ✔ on each fact, substitution on each edge). Forward chaining is **bottom-up** (facts first). One line to define each: BC = goal-driven, FC = data-driven.

REMEMBER IT

"Likes ← Food ← Eats + Alive." Bill is the only one with both facts, so x = Bill. Apples, Chicken and Sue are distractions.

Q6. Identify Task (T), Performance (P) and Experience (E) for a given learning problem (well-posed learning problem). IMPORTANT TOPIC

Why: scenario-based, fits sir's hint and CO3 ("fundamental understanding of ML"). Quick 5 marks.

DEFINITION (TOM MITCHELL)

A program learns from **experience E** with respect to a class of **tasks T** and **performance measure P** if its performance at T, as measured by P, improves with E. A problem is well-posed when T, P and E are all clearly identified.

Problem	T	P	E
Checkers	Playing checkers	% games won	Games played against itself
Handwriting recognition	Recognise handwritten words in images	% words correctly classified	Database of labelled handwritten words
Robot driving	Drive on 4-lane highways using vision	Average distance before an error	Images + steering commands recorded from a human driver
Spam filter	Classify emails as spam / not	% correctly classified	Emails labelled by users
Movie recommender	Recommend movies a user will like	% recommendations watched / rated high	User's past ratings and watch history
Crop-disease detector	Identify disease from leaf photos	% correct diagnoses	Labelled leaf images from farmers

REMEMBER IT

T = verb (what it does), **P = percentage/number** (how we score it), **E = data** (what it learns from). If your P isn't a number, it's wrong.

Q7. Apply Find-S to find the maximally specific hypothesis (concept learning).

IMPORTANT TOPIC

Why: CO3 last year explicitly said "Concept learning". It's a table problem (not pseudocode), so it fits "problem solving".

Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
Sunny	Warm	Normal	Strong	Warm	Same	**Yes**
Sunny	Warm	High	Strong	Warm	Same	**Yes**
Rainy	Cold	High	Strong	Warm	Change	**No**
Sunny	Warm	High	Strong	Cool	Change	**Yes**

h0 = ⟨Ø, Ø, Ø, Ø, Ø, Ø⟩ (most specific)

x1 (+): h1 = ⟨Sunny, Warm, Normal, Strong, Warm, Same⟩ — copy it

x2 (+): Humidity differs → ? h2 = ⟨Sunny, Warm, ?, Strong, Warm, Same⟩

x3 (−): ignored (Find-S skips negatives)

x4 (+): Water, Forecast differ → h4 = **⟨Sunny, Warm, ?, Strong, ?, ?⟩**

Limitations (1 mark): ignores negative examples; can't detect noisy data; gives only one hypothesis (Candidate-Elimination gives the whole version space: S = {⟨Sunny, Warm, ?, Strong, ?, ?⟩}, G = {⟨Sunny, ?, ?, ?, ?, ?⟩, ⟨?, Warm, ?, ?, ?, ?⟩}).

REMEMBER IT

"Copy the first Yes, then Question-mark the differences, Skip the No's." (C-Q-S)

UNIT II — ALSO BE READY FOR (STUDY THE TOPIC)

English → FOL (∀ with ⇒, ∃ with ∧) · unification / MGU of two expressions · forward chaining "West is a criminal" · Bayesian network joint probability P = ∏ P(node | parents) · knowledge graph RDF triple (subject–predicate–object) and KG vs ontology.

III Unit III — Classification & Regression (CR's list first)

YOUR CR'S 3 QUESTIONS (TEACHER SAID TO STUDY THESE FOR CIE-1)

1. Difference between classification and regression. 2. Naive Bayes classifier. 3. Explanation of linear regression with example.

Q1. Differentiate between classification and regression with examples.

SURE SHOT — CR

Both are **supervised learning** (trained on labelled data). **Classification** predicts a discrete category / class label ("Which class?"), e.g. spam / not spam. **Regression** predicts a continuous numeric value ("How much?"), e.g. house price.

Basis	Classification	Regression
Output	Discrete label (+ class probability)	Continuous real number
Question	Which category?	How much / how many?
Example (students)	Pass / Fail	Marks scored
Example (health)	Disease present / absent	Blood-glucose level
Example (real estate)	Budget / mid / premium	Selling price
Algorithms	Logistic regression, Naive Bayes, SVM, decision tree, KNN	Linear, polynomial regression, SVR, regression tree
Loss	Cross-entropy, hinge	Squared / absolute error
Metrics	Accuracy, precision, recall, F1, confusion matrix, ROC-AUC	MAE, MSE, RMSE, R^2
Picture	Decision boundary separating classes	Best-fit line / curve through points
Threshold	Converts probability → class (e.g. 0.5)	Not needed

BONUS LINE EXAMINERS LIKE

Logistic regression is a **classification** algorithm despite its name — it outputs a probability (0–1) that a threshold turns into a class.

REMEMBER IT

"Can I average two answers?" Average of ₹40 L and ₹60 L = ₹50 L ✔ → regression. Average of "cat" and "dog" ✘ → classification. **Units (₹, kg, °C, marks) → regression.**

Q2. Explain the Naive Bayes classifier with an example.

SURE SHOT — CR

DEFINITION (1 MARK)

A supervised **probabilistic classifier** based on **Bayes' theorem** that predicts the class with the highest posterior probability. "**Naive**" because it assumes the features are conditionally independent given the class.

$$P(C \mid X) = \frac{P(X \mid C)\,P(C)}{P(X)} \quad \Rightarrow \quad \hat{C} = \operatorname{argmax}_C P(C) \prod_i P(x_i \mid C) \quad (P(X) \text{ is common} \rightarrow \text{dropped})$$

Email	"offer"	"meeting"	Class
E1–E3	Yes	No	Spam
E4	Yes	Yes	Spam
E5	No	No	Spam
E6–E8	No	Yes	Not
E9	Yes	Yes	Not
E10	No	No	Not

New email: offer = Yes, meeting = No
Priors: P(Spam) = P(Not) = 5/10 = 0.5
Spam: P(offer=Y) = 4/5, P(meeting=N) = 4/5
Not: P(offer=Y) = 1/5, P(meeting=N) = 1/5
Score(Spam) = 0.5 × 0.8 × 0.8 = 0.32
Score(Not) = 0.5 × 0.2 × 0.2 = 0.02
⇒ **Spam**, P = 0.32/0.34 = 94.12%

Zero-frequency + Laplace smoothing (1 mark): if a feature never occurs with a class its probability is 0 and wipes out the product → use (count + 1)/(class count + k). Here: 5/7 and 2/7 → Spam 86.21%. **Types:** Gaussian, Multinomial, Bernoulli, Categorical. **Pros:** simple, fast, good for text, works with little data. **Cons:** independence assumption, zero frequency, overconfident probabilities.

REMEMBER IT

"Prior × Likes × Likes … pick the biggest." 5 steps: **P-L-M-C-N** = Priors → Likelihood table → Multiply → Compare → Normalize. "Naive" = treats every word as a stranger to the others.

Q3. Explain linear regression with an example. SURE SHOT — CR

Linear regression is a supervised method that models a continuous dependent variable y as a straight-line function of an independent variable x: $\hat{y} = b_0 + b_1 x$ (b_0 = intercept, b_1 = slope). The best line minimises the sum of squared errors (least squares):

$$b_1 = \frac{\Sigma xy - (\Sigma x)(\Sigma y)/n}{\Sigma x^2 - (\Sigma x)^2/n} \qquad b_0 = \bar{y} - b_1 \bar{x}$$

Hours x	Marks y	x^2	xy
1	40	1	40
2	50	4	100
3	65	9	195
4	70	16	280
5	85	25	425
15	**310**	**55**	**1040**

$\bar{x} = 3$, $\bar{y} = 62$
$S_{xy} = 1040 - 15 \times 310/5 = 110$
$S_{xx} = 55 - 225/5 = 10$
$b_1 = 110/10 = 11$, $b_0 = 62 - 33 = 29$
$\hat{y} = 29 + 11x$ → +11 marks per study hour
Predict 6 hours: 29 + 66 = 95
SSE = 20, SST = 1230 → $R^2 = 1 - 20/1230 = 0.98$ (excellent fit)

Uses: sales forecasting, price prediction, risk assessment. **Assumptions:** linear relationship, independent errors, constant variance. **Limitations:** outliers, non-linear data, unreliable extrapolation. Draw the scatter plot with the line (1 mark).

REMEMBER IT

"**Slope = how much y moves per x; intercept = where it starts.**" Table columns to draw: **x | y | x^2 | xy** ("X-Y-Square-Product"). The formula's numerator uses xy, the denominator uses x^2.

Q4. Scenario: classify each problem as classification or regression and name a suitable algorithm. IMPORTANT TOPIC

Scenario	Type	Algorithm
Predict tomorrow's maximum temperature	Regression	Linear regression
Detect whether a transaction is fraud	Classification (binary)	Logistic regression / Naive Bayes
Predict a student's CGPA from attendance and hours	Regression	Multiple linear regression
Tumour benign or malignant	Classification	SVM / logistic regression
Estimate a car's resale price	Regression	Linear / polynomial regression
Classify news into sports / politics / tech	Classification (multiclass)	Naive Bayes (multinomial)
Predict number of app downloads next month	Regression	Linear regression
Will a customer churn (yes/no)?	Classification	Logistic regression / decision tree

TIP

Justify each in one phrase: "output is a number" or "output is a category". That phrase is where the mark is.

Q5. Naive Bayes numerical on a new dataset (PlayTennis / stolen car type). IMPORTANT TOPIC

If NB comes as a problem instead of theory, it will be this format.

Stolen car: 10 cars, 5 stolen. Classify (Red, SUV, Domestic).
Yes: Red 3/5, SUV 1/5, Domestic 2/5. No: Red 2/5, SUV 3/5, Domestic 3/5.

$$P(\text{Yes})\cdot\ldots = 0.5 \times 3/5 \times 1/5 \times 2/5 = 0.024$$
$$P(\text{No})\cdot\ldots = 0.5 \times 2/5 \times 3/5 \times 3/5 = 0.072$$
$\Rightarrow$ **Not stolen**

PlayTennis (14 days, 9 yes / 5 no). Classify (Sunny, Cool, High, Strong).

$$\text{Yes: } 9/14 \times 2/9 \times 3/9 \times 3/9 \times 3/9 = 0.0053$$
$$\text{No: } 5/14 \times 3/5 \times 1/5 \times 4/5 \times 3/5 = 0.0206$$
$\Rightarrow$ **No** $(P = 0.0206/0.0259 = 0.795)$

EXAM LAYOUT (GETS FULL MARKS)

(1) Prior table · (2) likelihood table only for the values in the new instance · (3) multiply for each class · (4) compare and box the class · (5) normalize. Write fractions first, decimals at the end — fewer rounding errors.

Q6. A shop's ad spend (x, ₹ thousands) and sales (y, ₹ lakhs) are given. Fit a regression line, interpret it and predict. IMPORTANT TOPIC

x	y	x − x̄	y − ȳ	(x−x̄)(y−ȳ)	(x−x̄)²
1	3	−2	−0.6	1.2	4
2	4	−1	0.4	−0.4	1
3	2	0	−1.6	0	0
4	4	1	0.4	0.4	1
5	5	2	1.4	2.8	4
x̄ = 3, ȳ = 3.6				**4**	**10**

$b_1 = 4/10 = 0.4$; $b_0 = 3.6 - 1.2 = 2.4$

$\hat{y} = 2.4 + 0.4x$

Interpretation: every extra ₹1000 of ads adds ₹0.4 lakh sales; with zero ads, sales ≈ ₹2.4 lakh.

Predict x = 6: 2.4 + 2.4 = ₹4.8 lakh (extrapolation)

$R^2 = \Sigma(\hat{y}-\bar{y})^2/\Sigma(y-\bar{y})^2 = 1.6/5.2 \approx 0.31$ → weak fit: ad spend explains only ~31% of sales variation.

REMEMBER R^2

R^2 = "**how much of the up-and-down in y the line explains**". Near 1 = line hugs the points; near 0 = points scattered.

FINAL 10-MINUTE CHECKLIST BEFORE ENTERING THE HALL

- ☐ PEAS of a self-driving car + environment type in one line
- ☐ A*: f = g + h; the 8-puzzle answer is 5 moves (U, U, L, D, R)
- ☐ Modus Ponens valid; affirming the consequent invalid; validity ≠ truth
- ☐ Curiosity chain: Dog → Lover → No-kill → Cat is Animal → Not Jack → Curiosity
- ☐ Teacher / Explorer / Gamer → S / U / R
- ☐ Bayes: G-B-T-A; the disease example gives ≈ 8.8%
- ☐ Classification = category, regression = number (+ the 10-row table)
- ☐ Naive Bayes: P-L-M-C-N; email → Spam 94.12%
- ☐ Linear regression: x | y | x² | xy → ŷ = 29 + 11x